



Průmyslová robotika II – Semestrální projekt

Mobilní aplikace pro ovládání robota klavíristy

Datum odevzdání: 7.12.2025

Vypracoval: Lukáš Janeček JAN0837, Tomáš Pernica PER0176

Obsah

Seznam obrázků.....	2
1 Úvod do problematiky – Rozbor zadání.....	3
2 React Native.....	3
2.1 Tok dat.....	6
2.2 Role	6
2.3 Ošetření stavů/scénářů	7
3 Závěr – zhodnocení výsledků.....	7
4 Zdroje.....	7

Seznam obrázků

Obrázek 1 - Blokové schéma systému	3
Obrázek 2 - Hlavní stránka	4
Obrázek 3 - Pop up hlavní stránky	4
Obrázek 4 - Admin stránka	5
Obrázek 5 - Pop up	5
Obrázek 6 - Piano stránka	5
Obrázek 7 - Skladby stránka.....	6
Obrázek 8 - Blokové schéma toku dat	6
Obrázek 9 - Blokové schéma toku dat při stisknutí klávesy	6
Obrázek 10 - Ukázka funkčnosti.....	7

1 Úvod do problematiky – Rozbor zadání

Po připojení uživatele na síť robota je uživatel po načtení QR kódu schopen otevřít webové prostředí, přes které může manuálně ovládat stisknutí jednotlivých kláves robotem.

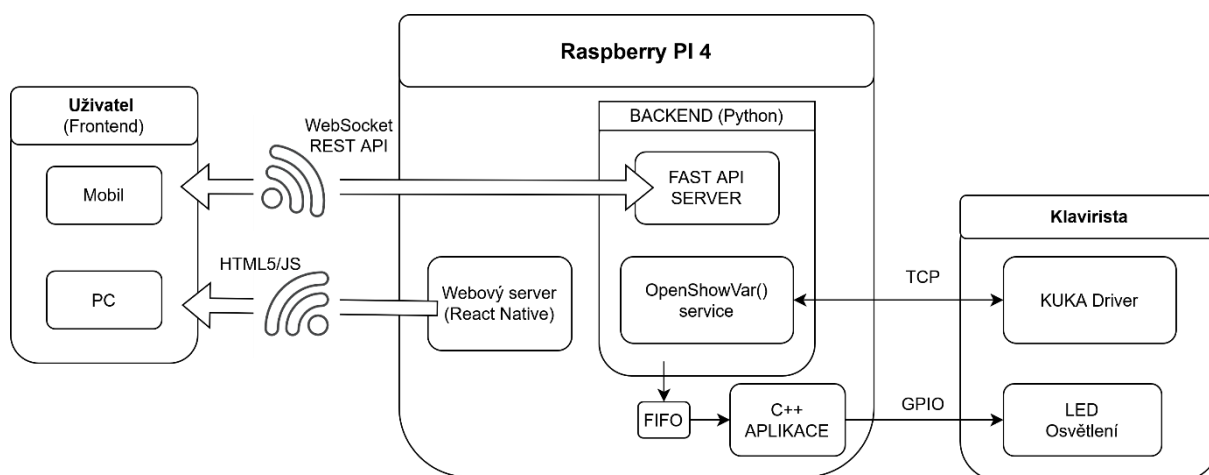
Celý systém je postaven na Raspberry Pi 4, které funguje jako centrální řídicí jednotka. Na Raspberry Pi běží backendová aplikace v Pythonu, poskytující REST API a zajišťující komunikaci mezi uživatelem a fyzickým systémem „Klavírista“. Uživatel přistupuje k systému buď z mobilního, nebo počítačového zařízení, a to prostřednictvím webového rozhraní nebo aplikace postavené na React Native. Frontend odesílá požadavky přes síťové rozhraní k backendu (REST API / HTTP), případně získává stavové informace opět ze serveru běžícího na Raspberry Pi.

Backend je rozdělen na dvě hlavní části. První část tvoří FASTAPI server, který zpracovává příchozí požadavky uživatele, provádí validaci, plánování akcí a poskytuje odpovědi zpět do frontendu. Druhou část backendu tvoří služba OpenShowVar(), která zajišťuje online komunikaci mezi Raspberry Pi a řídicími moduly robota KUKA. Backend podle potřeby předává instrukce do robota s využitím knihovny OpenShowVar díky, které pak následně komunikuje s robotem nebo dalšími podsystémy.

Pro komunikaci s hardwarem je v systému přítomna také C++ aplikace, která běží na Raspberry Pi a je propojena s backendem prostřednictvím FIFO sdílené paměti. C++ aplikace vykonává časově kritické nebo hardwarové operace, jako je ovládání externího LED osvětlení či doplňkových efektů.

Na straně zařízení klavírista se nachází dva hlavní moduly:

- KUKA Driver řídí pohyby samotného robotického ramene na základě dat přijímaných přes OpenShowVar.
- LED Osvětlení je samostatně řízené pomocí C++ aplikace na Raspberry Pi.



Obrázek 1 - Blokové schéma systému

2 React Native

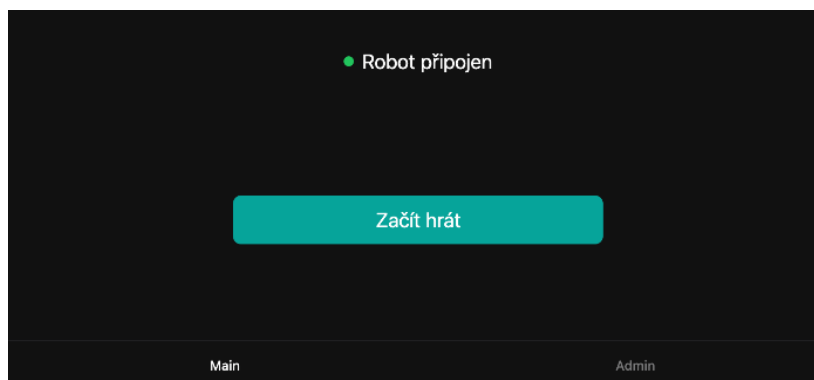
Při vývoji aplikace jsme vycházeli z defaultního template pro React Native aplikaci. V templatu jsme doplnili potřebné stránky, konfigurační soubory pro proměnné a obsluhy komunikace pro WebSocket a REST API.

Služba pro obsluhu WebSocketu (`hooks/useWebSocket.ts`) zajišťuje připojení se k WebSocketu běžícím na backendu. Po připojení uživatel dostane automaticky roli watchera. Služba také hlídá příchozí zprávy o změně role a uplatňuje touto změnu napříč frontendem.

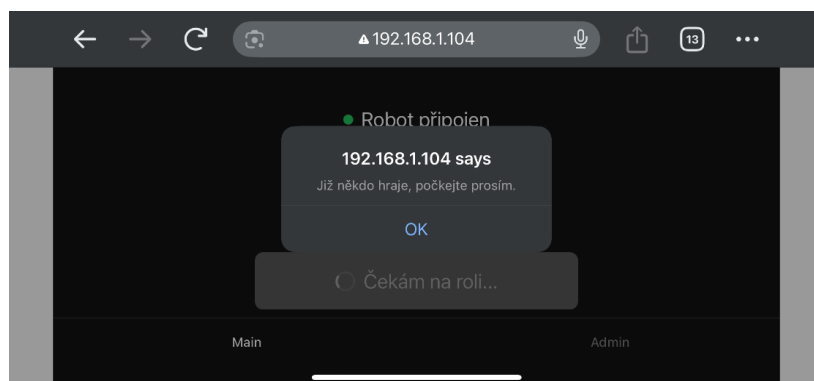
Služba pro periodické kontroly stavu připojení robota (`hooks/useRobotConnectionPolls.ts`) kontroluje zda je Raspberry PI připojeno k robotovi, aktuální stav robota (zda hraje píseň nebo klávesy z telefonu) a zda není překlemtá možnost ovládání pomocí telefonu či počítače.

Aplikace reaguje na velikost displeje spolu s možností otočení obrazovky na mobilních zařízeních. Styly stránek a jejich komponent jsme nadefinovali ve spodní části kódu ve třídě `styles` pro jednotlivé stránky aplikace, kdy se v každé komponentě na tyto styly odkazujeme. Na každé stránce se uživateli zobrazuje informace o konektivitě robota. Dále je zde možnost vzdát se role performerera a uvolnit tak roli dalšímu připojenému uživateli po stisknutí tlačítka „Ukončit hraní“.

Hlavní strana slouží jako hlavní vstup do aplikace. Představuje tak vstupní bod pro získání role interpreta. Backend zde vystavuje WebSocket endpointy, které spravují přidělování rolí v místnosti. Aplikace zde čeká na změnu role v rámci WebSocket přítomnosti. Jakmile backend odešle notifikaci o obdržení role interpreta uživatelem, tak hlavní stránka provede přesměrování na stránku pro piano, kde již probíhá vlastní ovládání. Dokud je uživatel na hlavní stránce, je jeho role watcher.

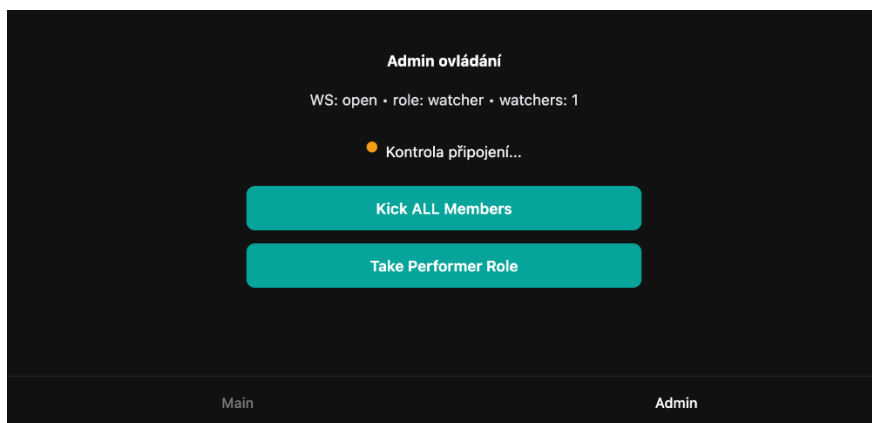


Obrázek 2 - Hlavní stránka

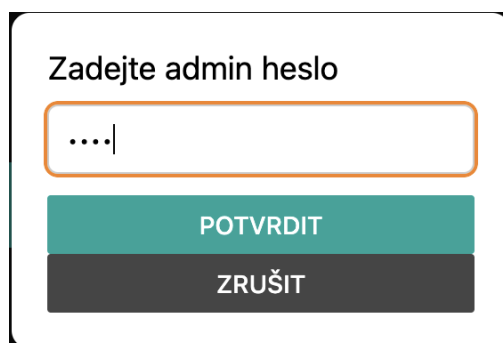


Obrázek 3 - Pop up hlavní stránky

Admin stránka slouží jako řídicí a diagnostický panel pro WebSocket. Uživatelům se zobrazují informace o roli, stavu WebSocketu, konektivitě robota a počtu účastníků. Backend zde zajišťuje posílání a přijímání informací o přítomnosti, rolích, stavu robota a událostech. Stránka umožňuje administrativní akce pro násilné převzetí práva interpreta a odpojení všech připojených zařízení, což u nich vyvolá přenačtení stránky a nové přiřazení rolí v místnosti.

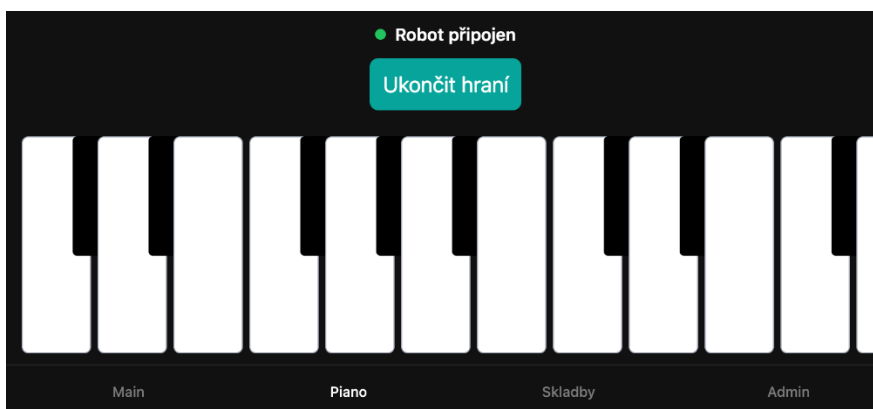


Obrázek 4 - Admin stránka



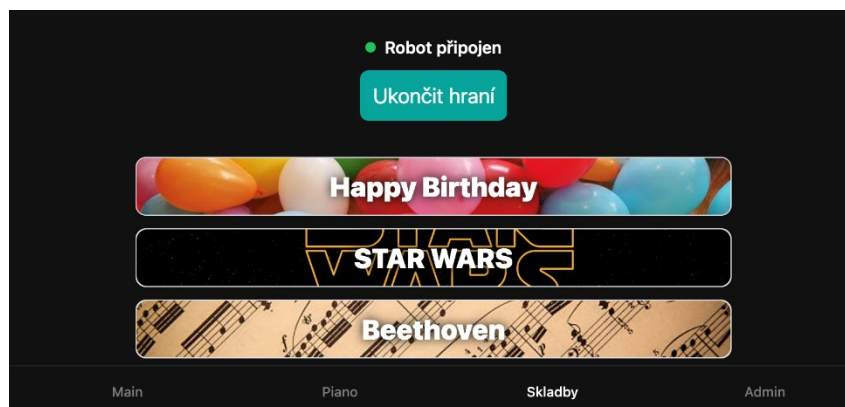
Obrázek 5 - Pop up

Pro stránku piano jsme si nadefinovali počet kláves, které je robot schopen bezpečně nahrát. Každé klávese jsme nadefinovali jedinečné ID, které předáváme dále po zmáčknutí. Při zmáčknutí také vyhodnocujeme délku zmáčknutí pro možnost hraní klávesy uživatelem po dobu stlačení klávesy. Stránka piano provádí periodické stahování stavu robota a udržuje propojení s backendem přes WebSocket.



Obrázek 6 - Piano stránka

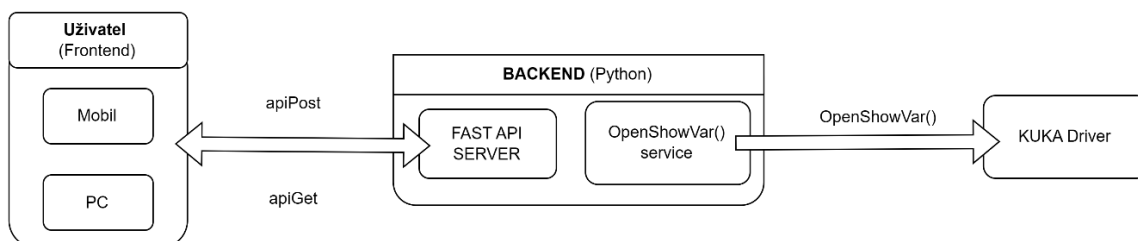
Stránka skladby poskytuje jednoduché rozhraní pro hraní předdefinovaných skladeb v robotovi. Stránka skladby také provádí periodické stahování stavu robota a udržuje tak spojení s backendem přes WebSocket.



Obrázek 7 - Skladby stránka

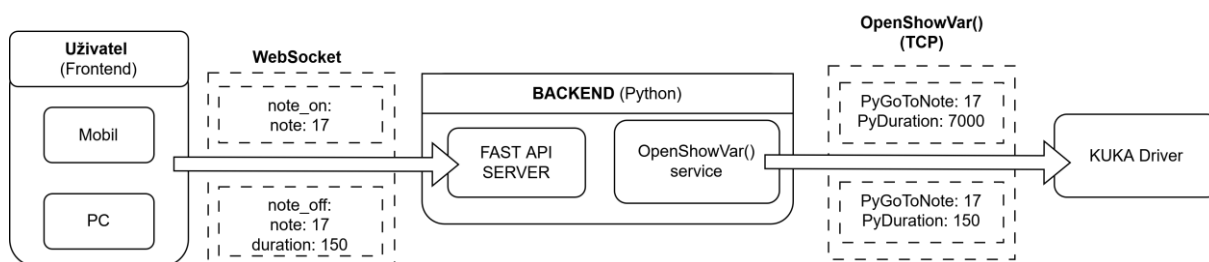
2.1 Tok dat

Celý systém je založen na komunikaci mezi uživatelským rozhraním (Frontend), serverovou částí (Backend – FastAPI + OpenShowVar služba) a samotným KUKA robotickým driverem. Uživatel může systém ovládat jak z mobilního, tak z počítačového webového rozhraní.



Obrázek 8 - Blokové schéma toku dat

Schéma toku dat při stisknutí klávesy znázorňuje, jak systém zpracuje konkrétní vstup r webového rozhraní, například stisknutí a uvolnění klávesy. Backend přijme požadavek a přeloží jej na odpovídající robotický povel, což reprezentuje pokyn hraj notu, jakou cílovou notu a definovanou délku prodlevy, kterou by měl robot dodržet. KUKA driver následně započne pohyb a fyzicky stiskne zvolenou klávesu na pianu. Po ukončení pohybu se robot vrací do výchozí pozice a opět čeká na příchozí pokyn.



Obrázek 9 - Blokové schéma toku dat při stisknutí klávesy

2.2 Role

V aplikaci rozlišujeme dvě role, watcher (pozorovatel) a performer (interpret). Je to z důvodu omezení práv připojeným uživatelům, aby nevznikala kolize v komunikaci a ovládání robota. Performer má práva na otevírání stránek a interagovat v nich. Watcher nemá práva na otevírání stránek, vidí pouze hlavní stránku a může pouze při neaktivitě performeru převzít status performeru. Pokud je někdo jiný performer a je aktivní, převzetí performeru je zakázáno a je vyobrazena hláška o zamítnutí převzetí.

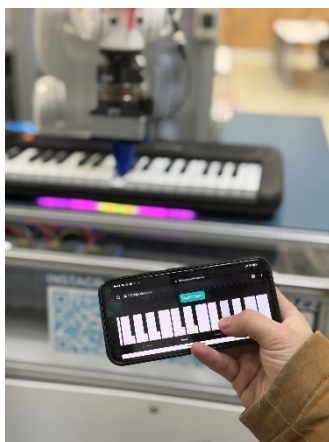
Může také otevřít admin stránku pro vyhrazení práv performerů, v případě zadání příslušného hesla do popupu pro otevření samotné stránky.

2.3 Ošetření stavů/scénářů

Aby byl zajištěn hladký průběh ovládání robota jsou jednotlivé stránky a funkce opatřeny ověřováním, zda se operace provedly správně. Dokud není rozhodnuto o převzetí role watchera nebo performerů je v rámci tlačítka vyobrazeno načítání. Pokud uživatel performer více než 15 sekund neintegruje s aplikací má watcher možnost si práva uzmout pro sebe a performerů tak přepnout do stavu watchera. Pokud není uživatel aktivní a jiný uživatel nežádá o převzetí performerů, robot přejde zpět do idle módu. V tento moment není připraven na hraní, avšak klávesy jsou stisknutelné pro opětovné uvedení robota do hrací pozice. Po přepnutí na piano stránku se robot přesune do výchozí pozice pro hraní. Dokud není robot připraven není možné mačkat klávesy. Rovněž na stránce skladby po spuštění přehrávání skladby je interakce s pianem a tlačítka skladeb zakázána.

3 Závěr – zhodnocení výsledků

Pro vývoj, debug a sdílení souborů v rámci týmu jsme použili GitHub, kam jsme nahrávali naše změny a postupně dále rozvíjeli naše řešení. Finální řešení je optimalizováno pro mobilní a počítačové zařízení. Pro displej na klavíristovi jsme dodali dva QR kódy, jeden pro připojení k Wi-fi síti robota a druhý pro otevření aplikace v prohlížeči daného zařízení.



Obrázek 10 - Ukázka funkčnosti

4 Zdroje

[1] <https://reactnative.dev/docs/getting-started-without-a-framework>

[2] <https://github.com/LukiJanecek/AdvancedPianistRobot>